최적화 기법 분석

개요

libiop 라이브러리는 대규모 zkSNARK 증명 생성과 검증을 실용적으로 만들기 위해 다양한 최적화 기법을 적용합니다. 이 문서에서는 메모리 풀링, 캐시 효율성, 배치 처리, 벡터화, 플랫폼별 최적화 등 핵심 최적화 전략들을 상세히 분석합니다.

메모리 풀링과 캐시 효율성 개선

메모리 풀링 전략

벡터 메모리 풀

대용량 벡터 연산에서 메모리 할당/해제 오버헤드를 줄이기 위한 풀링:

```
template<typename FieldT>
class field_vector_pool {
private:
std::vector<std::vector<FieldT>> available_vectors_;
std::mutex pool_mutex_;
std::map<size_t, std::queue<std::vector<FieldT>>> size_buckets_;
public:
std::vector<FieldT> acquire(size_t  required_size) {
std::lock_guard<std::mutex> lock(pool_mutex_);
```

```
// 적절한 크기 버킷에서 벡터 찾기

auto bucket_it = size_buckets_.lower_bound(required_size);

if (bucket_it != size_buckets_.end() && !bucket_it->second.empty()) {

auto vec = std::move(bucket_it->second.front());

bucket_it->second.pop();

vec.resize(required_size);

return vec;

}

// 새 벡터 생성

std::vector<FieldT> new_vec;

new_vec.reserve(next_power_of_2(required_size));

new_vec.resize(required_size);

return new_vec;

}

void  release(std::vector<FieldT>&&  vec) {

std::lock_guard<std::mutex> lock(pool_mutex_);

const  size_t capacity = vec.capacity();

vec.clear(); // 크기는 0으로, 용량은 유지

size_buckets_[capacity].push(std::move(vec));

}

};
```

다항식 계수 풀링

다항식 연산에서 임시 계수 벡터 재사용:

```
template<typename FieldT>
class polynomial_coefficient_pool {
private:
thread_local static std::vector<std::vector<FieldT>> coefficient_cache_;
static constexpr size_t MAX_CACHE_SIZE = 16;
public:
static std::vector<FieldT> get_coefficients(size_t degree_bound) {
if (!coefficient_cache_.empty()) {
auto coeffs = std::move(coefficient_cache_.back());
coefficient_cache_.pop_back();
coeffs.resize(degree_bound + 1, FieldT::zero());
return coeffs;
}
return std::vector<FieldT>(degree_bound + 1, FieldT::zero());
}
static void return_coefficients(std::vector<FieldT>&& coeffs) {
if (coefficient_cache_.size() < MAX_CACHE_SIZE) {
coeffs.clear();
coefficient_cache_.push_back(std::move(coeffs));
}
}
```

```
};
```

캐시 효율성 최적화

데이터 지역성 개선

메모리 접근 패턴을 최적화하여 캐시 미스 최소화:

```
template<typename FieldT>
void cache_friendly_matrix_multiply(
const std::vector<std::vector<FieldT>>& A,
const std::vector<FieldT>& x,
std::vector<FieldT>& result) {
const size_t rows = A.size();
const size_t cols = x.size();
const size_t BLOCK_SIZE = 64; // L1 캐시 라인 크기에 맞춤
// 블록 단위 행렬-벡터 곱셈
for (size_t i = 0; i < rows; i += BLOCK_SIZE) {
const size_t i_end = std::min(i + BLOCK_SIZE, rows);
for (size_t j = 0; j < cols; j += BLOCK_SIZE) {
const size_t j_end = std::min(j + BLOCK_SIZE, cols);
// 내부 블록 계산
for (size_t ii = i; ii < i_end; ++ii) {
FieldT sum = FieldT::zero();
for (size_t jj = j; jj < j_end; ++jj) {
```

```
sum += A[ii][jj] * x[jj];

}

result[ii] += sum;

}

}

}

}
```

프리페치 최적화

메모리 프리페치를 통한 지연시간 숨김:

```
template<typename FieldT>

void  prefetch_optimized_polynomial_evaluation(

const std::vector<FieldT>&  coefficients,

const std::vector<FieldT>&  evaluation_points,

std::vector<FieldT>&  results) {

const  size_t PREFETCH_DISTANCE = 8;

for (size_t i = 0; i < evaluation_points.size(); ++i) {

// 미래 데이터 프리페치

if (i + PREFETCH_DISTANCE < evaluation_points.size()) {

__builtin_prefetch(&evaluation_points[i + PREFETCH_DISTANCE], 0, 3);

}

// 호너의 방법으로 다항식 평가

const FieldT& x = evaluation_points[i];
```

```
FieldT result = coefficients.back();
for (int j = coefficients.size() - 2; j >= 0; --j) {
result = result * x + coefficients[j];
}
results[i] = result;
}
}
```

캐시 친화적 데이터 구조

```
template<typename FieldT>
class cache_optimized_sparse_matrix {
private:
// CSR (Compressed Sparse Row) 형식으로 저장
std::vector<FieldT> values_;
std::vector<size_t> column_indices_;
std::vector<size_t> row_pointers_;
public:
void multiply_vector(const std::vector<FieldT>& x, std::vector<FieldT>& resul
const size_t rows = row_pointers_.size() - 1;
#pragma omp parallel for
for (size_t i = 0; i < rows; ++i) {
FieldT sum = FieldT::zero();
```

```
const  size_t start = row_pointers_[i];

const  size_t end = row_pointers_[i + 1];

// 연속된 메모리 접근으로 캐시 효율성 극대화

for (size_t j = start; j < end; ++j) {

sum += values_[j] * x[column_indices_[j]];

}

result[i] = sum;

}

}

};
```

배치 처리와 벡터화 최적화

배치 처리 전략

FFT 배치 처리

여러 다항식의 FFT를 동시에 처리:

```
template<typename FieldT>

class batch_fft_processor {

private:

field_subset<FieldT> domain_;

size_t batch_size_;

public:
```

```
std::vector<std::vector<FieldT>> process_batch(
const std::vector<std::vector<FieldT>>&  polynomial_batch) {
const  size_t num_polynomials = polynomial_batch.size();
std::vector<std::vector<FieldT>> results(num_polynomials);
// 배치 단위로 병렬 처리
#pragma  omp  parallel  for
for (size_t i = 0; i < num_polynomials; ++i) {
results[i] = FFT_over_field_subset(polynomial_batch[i], domain_);
}
return results;
}
// 전치된 배치 처리 (메모리 접근 최적화)
std::vector<std::vector<FieldT>> process_transposed_batch(
const std::vector<std::vector<FieldT>>&  transposed_batch) {
const  size_t domain_size = domain_.num_elements();
const  size_t num_polynomials = transposed_batch[0].size();
std::vector<std::vector<FieldT>> results(num_polynomials);
for (auto& result : results) {
result.resize(domain_size);
}
// 도메인 원소별로 모든 다항식 동시 평가
#pragma  omp  parallel  for
for (size_t domain_idx = 0; domain_idx < domain_size; ++domain_idx) {
```

```
const FieldT eval_point = domain_.element_by_index(domain_idx);

for (size_t poly_idx = 0; poly_idx < num_polynomials; ++poly_idx) {

results[poly_idx][domain_idx] =

evaluate_polynomial_at_point(transposed_batch, poly_idx, eval_point);

}

}

return results;

}

};
```

머클 트리 배치 구성

여러 오라클의 머클 트리를 배치로 구성:

```
template<typename FieldT, typename hash_digest_type>

class batch_merkle_tree_builder {

private:

std::shared_ptr<leafhash<FieldT, hash_digest_type>> leaf_hasher_;

two_to_one_hash_function<hash_digest_type> node_hasher_;

public:

std::vector<merkle_tree<FieldT, hash_digest_type>> build_batch(

const std::vector<std::vector<std::vector<FieldT>>>&  oracle_batches) {

const  size_t num_trees = oracle_batches.size();

std::vector<merkle_tree<FieldT, hash_digest_type>> trees;

trees.reserve(num_trees);
```

```
// 병렬로 머클 트리 구성
#pragma omp parallel for
for (size_t i = 0; i < num_trees; ++i) {
merkle_tree<FieldT, hash_digest_type> tree(
oracle_batches[i][0].size(), // num_leaves
leaf_hasher_,
node_hasher_,
digest_len_bytes_,
make_zk_,
security_parameter_);
tree.construct(oracle_batches[i]);
#pragma omp critical
{
trees.push_back(std::move(tree));
}
}
return trees;
}
};
```

벡터화 최적화

SIMD 명령어 활용

```
template<typename FieldT>
class simd_field_operations {
public:
// AVX2를 사용한 벡터화된 필드 덧셈 (GF(2^n) 필드용)
static void vectorized_add(
const std::vector<FieldT>& a,
const std::vector<FieldT>& b,
std::vector<FieldT>& result) {
const size_t n = a.size();
const size_t simd_width = 8; // AVX2는 256비트 = 8개의 32비트 원소
size_t i = 0;
// SIMD 처리 가능한 부분
for (; i + simd_width <= n; i += simd_width) {
#pragma omp simd
for (size_t j = 0; j < simd_width; ++j) {
result[i + j] = a[i + j] + b[i + j];
}
}
// 나머지 원소들 처리
for (; i < n; ++i) {
result[i] = a[i] + b[i];
}
```

```
}

// 벡터화된 스칼라 곱셈

static void vectorized_scalar_multiply(

const std::vector<FieldT>& vec,

const FieldT& scalar,

std::vector<FieldT>& result) {

const size_t n = vec.size();

#pragma omp simd

for (size_t i = 0; i < n; ++i) {

result[i] = vec[i] * scalar;

}

}

};
```

자동 벡터화 유도

컴파일러의 자동 벡터화를 최대한 활용:

```
template<typename FieldT>

void compiler_friendly_polynomial_multiply(

const std::vector<FieldT>& a,

const std::vector<FieldT>& b,

std::vector<FieldT>& result) {

const size_t deg_a = a.size() - 1;

const size_t deg_b = b.size() - 1;
```

```
const size_t result_degree = deg_a + deg_b;
result.assign(result_degree + 1, FieldT::zero());
// 컴파일러 벡터화 힌트
#pragma omp simd collapse(2)
for (size_t i = 0; i <= deg_a; ++i) {
for (size_t j = 0; j <= deg_b; ++j) {
result[i + j] += a[i] * b[j];
}
}
}
```

플랫폼별 최적화와 하드웨어 가속 활용

CPU 아키텍처별 최적화

x86-64 최적화

```
#ifdef __x86_64__
template<typename FieldT>
class x86_optimized_operations {
public:
// BMI2 명령어를 사용한 비트 조작 최적화
static uint64_t fast_bit_reverse(uint64_t x, int bits) {
#ifdef __BMI2__
```

```
return _pdep_u64(_pext_u64(x, (1ULL << bits) - 1),
bit_reverse_table[bits]);
#else
return generic_bit_reverse(x, bits);
#endif
}
// RDTSC를 사용한 고정밀 타이밍
static uint64_t get_cpu_cycles() {
uint32_t hi, lo;
__asm__ volatile ("rdtsc" : "=a"(lo), "=d"(hi));
return ((uint64_t)hi << 32) | lo;
}
};
#endif
```

ARM 최적화

```
#ifdef __aarch64__
template<typename FieldT>
class arm_optimized_operations {
public:
// NEON SIMD 명령어 활용
static void neon_vector_add(
```

```
const float32_t*  a,
const float32_t*  b,
float32_t*  result,
size_t  count) {
const  size_t simd_count = count & ~3; // 4의 배수로 정렬
for (size_t i = 0; i < simd_count; i += 4) {
float32x4_t va = vld1q_f32(&a[i]);
float32x4_t vb = vld1q_f32(&b[i]);
float32x4_t vr = vaddq_f32(va, vb);
vst1q_f32(&result[i], vr);
}
// 나머지 원소들 처리
for (size_t i = simd_count; i < count; ++i) {
result[i] = a[i] + b[i];
}
}
};
#endif
```

GPU 가속 지원

CUDA 기반 FFT 가속

```
#ifdef  CUDA_ENABLED
```

```
template<typename FieldT>
class cuda_fft_accelerator {
private:
cufftHandle plan_;
FieldT* d_input_;
FieldT* d_output_;
size_t max_size_;
public:
cuda_fft_accelerator(size_t  max_fft_size) : max_size_(max_fft_size) {
cudaMalloc(&d_input_, max_size_ * sizeof(FieldT));
cudaMalloc(&d_output_, max_size_ * sizeof(FieldT));
cufftPlan1d(&plan_, max_size_, CUFFT_C2C, 1);
}
std::vector<FieldT> compute_fft(const std::vector<FieldT>&  input) {
const  size_t n = input.size();
// 데이터를 GPU로 복사
cudaMemcpy(d_input_, input.data(), n * sizeof(FieldT),
cudaMemcpyHostToDevice);
// GPU에서 FFT 실행
cufftExecC2C(plan_,
reinterpret_cast<cufftComplex*>(d_input_),
reinterpret_cast<cufftComplex*>(d_output_),
CUFFT_FORWARD);
```

```
// 결과를 CPU로 복사

std::vector<FieldT> result(n);

cudaMemcpy(result.data(), d_output_, n * sizeof(FieldT),

cudaMemcpyDeviceToHost);

return result;

}

~cuda_fft_accelerator() {

cufftDestroy(plan_);

cudaFree(d_input_);

cudaFree(d_output_);

}

};

#endif
```

컴파일러 최적화 활용

프로파일 기반 최적화 (PGO)

```
// 프로파일링을 위한 계측 코드

class performance_profiler {

private:

std::map<std::string, std::pair<uint64_t, uint64_t>> function_stats_;

public:

void  record_function_call(const std::string&  func_name, uint64_t  cycles) {
```

```
auto& stats = function_stats_[func_name];

stats.first++; // 호출 횟수

stats.second += cycles; // 총 사이클 수

}

void generate_pgo_data() const {

std::ofstream pgo_file("profile.txt");

for (const auto& [name, stats] : function_stats_) {

pgo_file << name << " " << stats.first << " " << stats.second << "\n";

}

}

};


// 핫 패스 최적화를 위한 매크로

#define LIKELY(x) __builtin_expect(!!(x), 1)

#define UNLIKELY(x) __builtin_expect(!!(x), 0)


template<typename FieldT>

FieldT optimized_field_inverse(const FieldT& x) {

if (LIKELY(!x.is_zero())) {

return x.inverse();

} else {

throw std::invalid_argument("Cannot invert zero element");

}
```

```
}
```

링크 타임 최적화 (LTO)

```
# CMakeLists.txt에서 LTO 활성화
if(CMAKE_BUILD_TYPE STREQUAL "Release")
set_property(TARGET libiop PROPERTY INTERPROCEDURAL_OPTIMIZATION TRUE)
# 추가 최적화 플래그
target_compile_options(libiop PRIVATE
-O3
-march=native
-mtune=native
-flto
-ffast-math
-funroll-loops
)
endif()
```

메모리 대역폭 최적화

메모리 접근 패턴 최적화

```
template<typename FieldT>
class memory_bandwidth_optimizer {
public:
```

```
// 스트리밍 저장을 사용한 캐시 우회
static void streaming_copy(
const std::vector<FieldT>& src,
std::vector<FieldT>& dst) {
const size_t n = src.size();
const FieldT* src_ptr = src.data();
FieldT* dst_ptr = dst.data();
#pragma omp parallel for
for (size_t i = 0; i < n; i += 64) { // 캐시 라인 크기
const size_t end = std::min(i + 64, n);
for (size_t j = i; j < end; ++j) {
// 논템포럴 저장 (캐시를 오염시키지 않음)
_mm_stream_si32(reinterpret_cast<int*>(&dst_ptr[j]),
*reinterpret_cast<const int*>(&src_ptr[j]));
}
}
_mm_sfence(); // 메모리 펜스
}
// NUMA 인식 메모리 할당
static std::vector<FieldT> numa_aware_allocate(size_t size) {
std::vector<FieldT> vec;
vec.reserve(size);
#ifdef NUMA_ENABLED
```

```
// 현재 스레드가 실행 중인 NUMA 노드에 메모리 할당
numa_set_localalloc();
vec.resize(size);
numa_set_interleave_mask(numa_all_nodes_ptr);
#else
vec.resize(size);
#endif
return vec;
}
};
```

이러한 최적화 기법들을 통해 libiop는 대규모 zkSNARK 증명을 실용적인 시간 내에 생성하고 검증할 수 있는 성능을 달성합니다. 특히 메모리 계층 구조를 고려한 최적화와 현대 CPU의 벡터화 기능 활용이 핵심적인 역할을 합니다.